CSE 258 Assignment 2
Anime Recommendation

Rhea Senthil Kumar



Agenda



1. Predictive Task

- How model will be evaluated
- Relevant baselines for comparison
- Assess validity of model's predictions

2. Exploratory Analysis, Data Collection, & Pre-Processing

- Data context, where it's from, what it's for
- Analysis results
- Data pre-processing

3. Modelling

- Formulating task as ML problem and models
- Advantages/ Disadvantages of model choices
- Architectural choices, implementation details

4. Evaluation

- How task should be evaluated + chosen metrics
- Baseline comparison
- Evaluation protocol + results

5. Discussion of Related Work

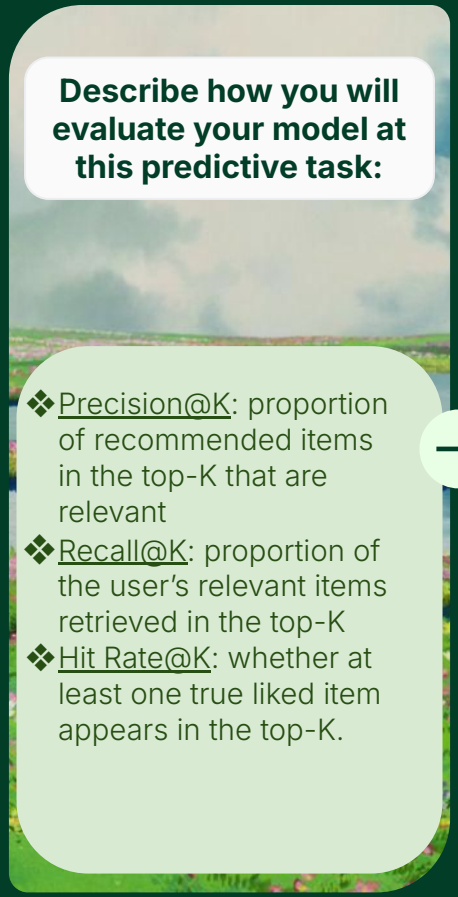
- Previous use of dataset
- Prior work
- Result comparison to prior work

6. Summary

- Predictive task + chosen model
- Key Results
- Reflection

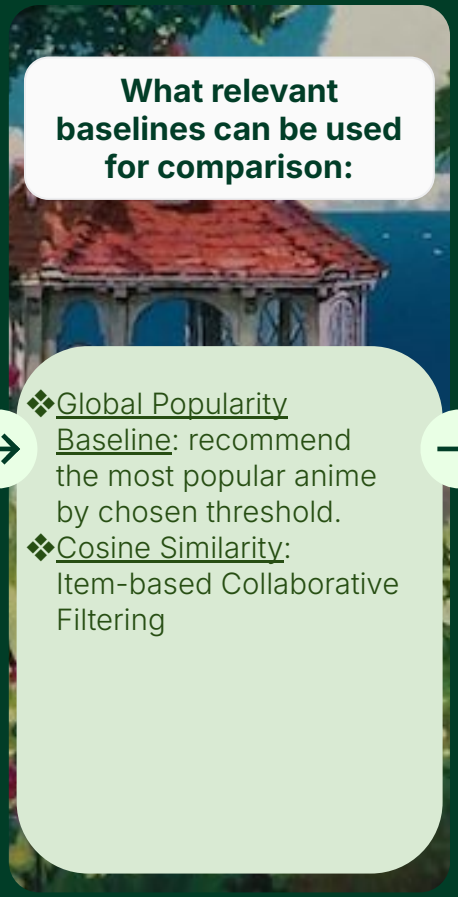
1. Predictive Task

Top-K personalized anime recommendation: Given a user's historical interactions (ratings, watches, or implicit liked signals), the goal is to rank all unseen anime and return the top-K items the user is most likely to enjoy using a Hybrid Bayesian Personalized Ranking (BPR) Model.



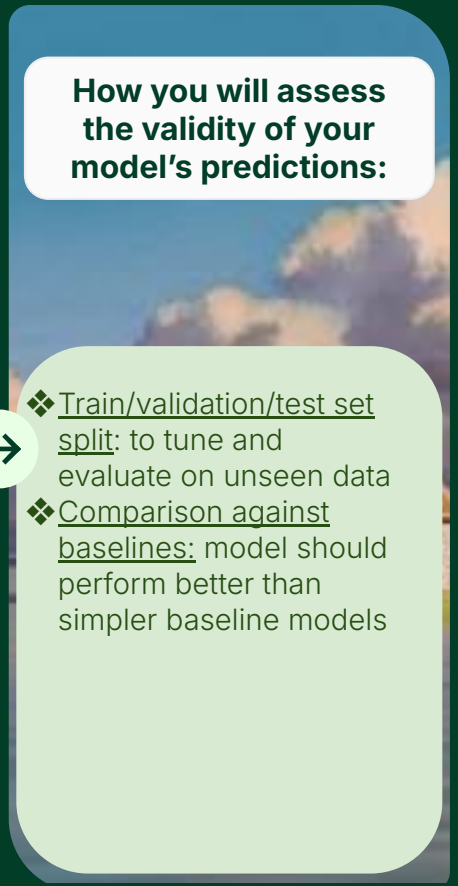
Describe how you will evaluate your model at this predictive task:

- Precision@K: proportion of recommended items in the top-K that are relevant
- Recall@K: proportion of the user's relevant items retrieved in the top-K
- Hit Rate@K: whether at least one true liked item appears in the top-K.



What relevant baselines can be used for comparison:

- Global Popularity Baseline: recommend the most popular anime by chosen threshold.
- Cosine Similarity: Item-based Collaborative Filtering



How you will assess the validity of your model's predictions:

- Train/validation/test set split: to tune and evaluate on unseen data
- Comparison against baselines: model should perform better than simpler baseline models

2. Exploratory Analysis, Data Collection, & Pre-Processing

Where does your dataset come from? What is it for, how was it collected, etc.?

Chosen Dataset: Anime Recommendation Database 2020, Kaggle

- https://www.kaggle.com/datasets/hernan4444/anime-recommendation-database-2020
- Contains recommendation data from ~310,0000 users and ~17,000 animes
- Includes 109 million user-anime interactions with watching statuses and ratings
- Data: user anime watching status, completed anime ratings, anime metadata (genre, studio, reviews, etc.)

Dataset Purpose:

- Experiment with recommendation systems without live scraping request delays
- Test recommendation models, such as collaborative filtering or content-based models
- Identify which data feature allows for the best anime recommendation system

How Data was Collected:

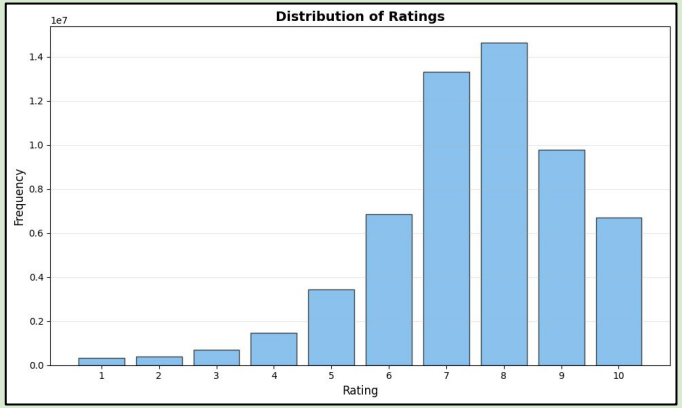
- data was scraped from MyAnimeList (MAL)
- Website scraping source: https://github.com/Hernan4444/MyAnimeList-Database

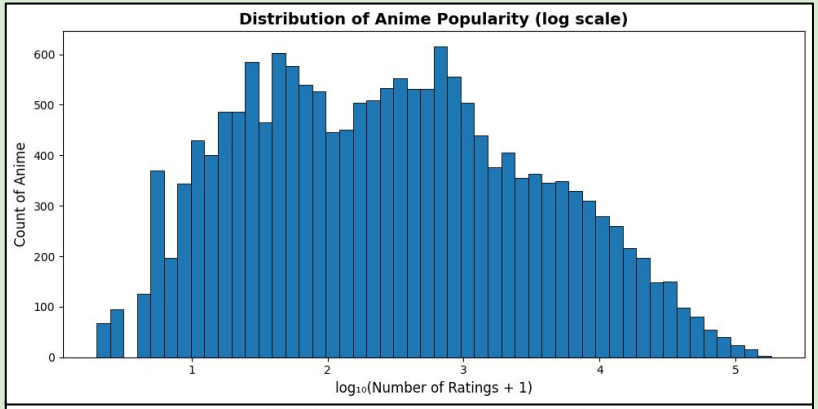


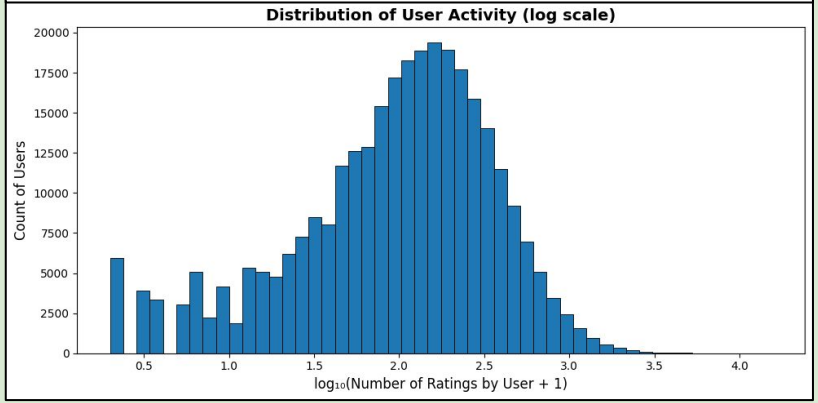
2. Exploratory Analysis, Data Collection, & Pre-Processing

Support your analysis with tables, plots, statistics, etc.







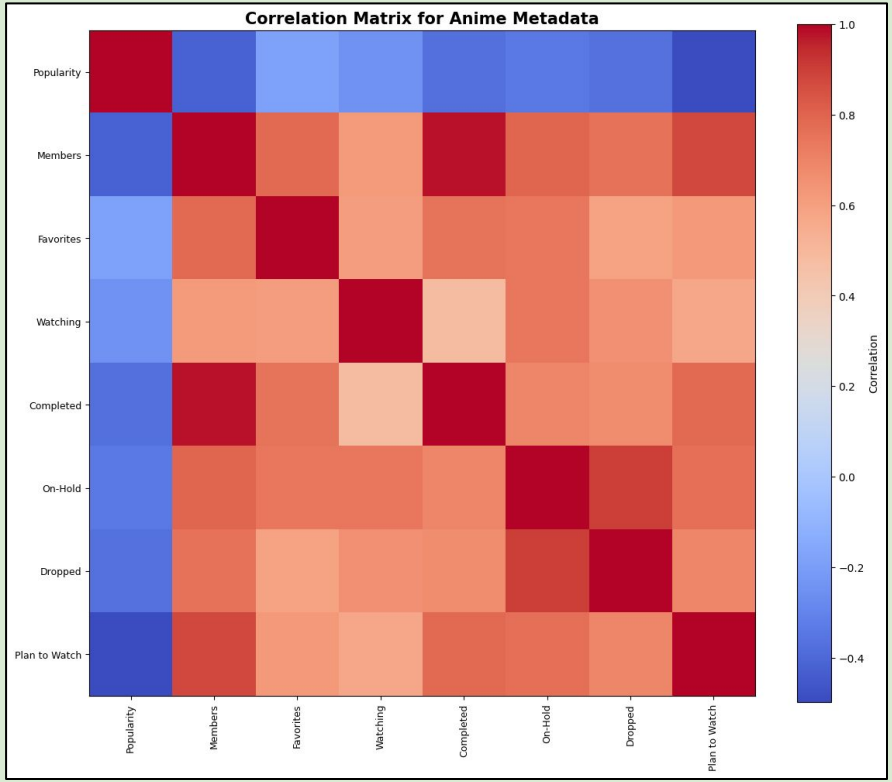


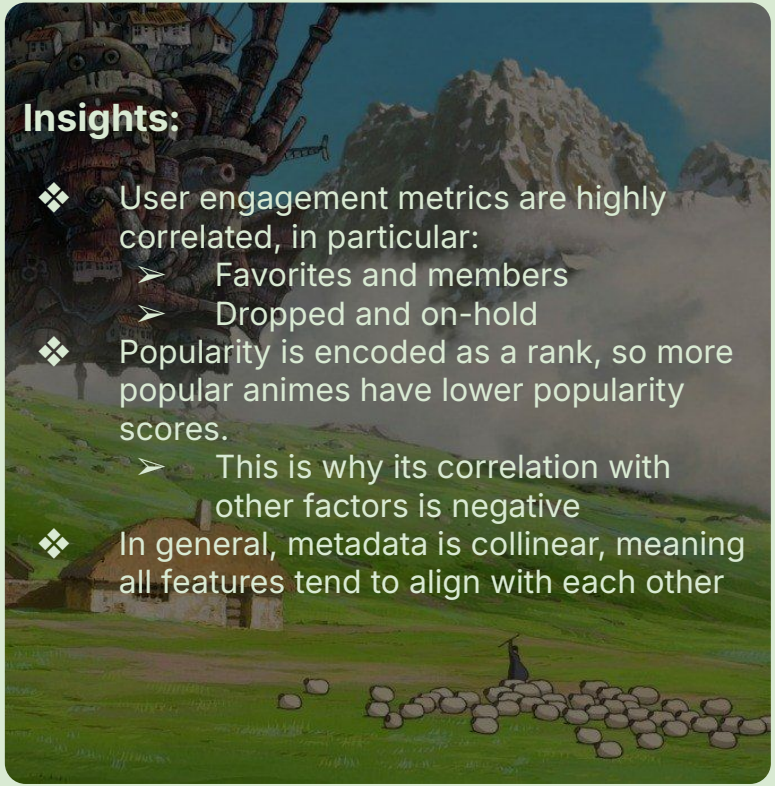
- Rating distribution heavily right-skewed
- Distribution of ratings for each anime generally unimodal, slightly left-skewed
- Distribution of user activity generally unimodal with many outliers to the left

→ These three plots ultimately helped to determine which thresholds to use to clean, remove, and pre-process data.

2. Exploratory Analysis, Data Collection, & Pre-Processing

Support your analysis with tables, plots, statistics, etc.





2. Exploratory Analysis, Data Collection, & Pre-Processing

Report how you processed the data (or how it was already processed)

1. Using distributions from exploratory work, remove sparse user/anime data:
   - Num. users: 293,842
   - Num. anime: 16,018

2. Encode user/anime pairs, convert datatype of anime genre and remove sparse genres

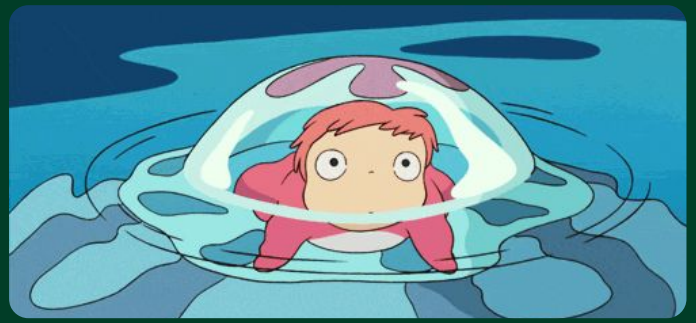
3. Process anime metadata: one-hot encodings, normalize features

4. Shuffle data and split into train, validation, and test sets:
   - Train: 80% (~46,000,000)
   - Validation: 10% (~5,700,000)
   - Test: 10% (~5,700,000)

5. Create dictionary of users to animes for negative sampling

6. Save preprocessed data locally to numpy arrays to avoid costly reloading/data manipulation

Referenced from Kaggle/related literature:
- Sparse user/anime/genre removal
- One-hot encoding of metadata and genre



3. Modelling

How do you formulate your task as an ML problem, e.g. what are the inputs, outputs, and what is being optimized? What models are appropriate for the task?

Goal: Predict which top-K animes that a user is most likely to watch.

Task Formulation:

- Implicit-feedback recommendation problem, where the training data consists of users rating animes
- Data consists of positive interactions between users and animes, as well as anime metadata

Inputs:

- User-item interaction matrix R
- User history
- Item-item similarity for collaborative filtering
- Global item statistics: mean rating, global popularity
- User statistics: average user rating
- Anime metadata

Outputs:

- Model outputs list of animes for each user, ranked by confidence

What is Optimized:

- Pairwise ranking objective with features about users/items and implicit feedback

$$f(u,i) = \alpha + \beta_u + \beta_i + (\gamma_u + \frac{1}{\|N(u)\|}\sum_{a \in N(u)} \rho_a) \cdot \gamma_i$$

Appropriate Model:

- Hybrid Implicit BPR Model with baseline fallbacks for cold start



3. Modelling

Discuss the advantages and disadvantages of different modeling approaches (complexity, efficiency, challenges in implementation, etc.)

Increasing Complexity

1 Popularity-Based & User/Item Average Model

Advantages:
- Good for cold start problems
- Fast, computationally efficient

Disadvantages:
- Not personalized for each user
- Not good for less popular animes

2 Item-Based Collaborative Filtering

Advantages:
- Simple, compare similarity
- Interpretable for similar users/animes

Disadvantages:
- Doesn't generalize beyond seen items
- Similarity matrix expensive for large datasets

3 Implicit BPR

Advantages:
- Optimizes ranking with loss function
- Can handle latent information

Disadvantages:
- Training is more computationally expensive
- Not good with cold start problems

4 Hybrid BPR

Advantages:
- Can personalize to each user
- Can handle cold start problems

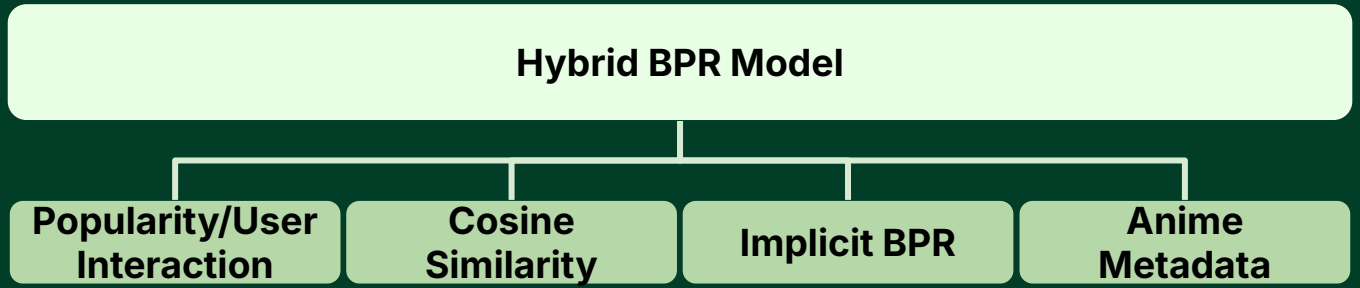
Disadvantages:
- Complex implementation (several components)
- Expensive training

Chosen Model

3. Modelling

Walk through your code, explaining architectural choices and any implementation details.

Hybrid BPR is a weighted combination of several models:



Architecture:

- ❖BPR has most weight, followed by cosine similarity, then anime metadata, user interaction, and popularity baselines
  - ➢More powerful components can learn implicit relationships
  - ➢For sparse data, use simpler models as fallback
- ❖Normalizes scores from each component to be added together
- ❖Uses content-based and collaborative filtering to generate top-K recommendations



4. Evaluation

How should your task be evaluated? Can you justify why your particular metrics are more appropriate than others?



We aim to rank relevant anime higher in each user's top-K recommendations → **evaluation metrics must measure how well the model retrieves relevant items from each user's held-out interactions in the test set.**

Three Evaluation Metrics Selected:

Precision@K

$$\text{precision} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{retrieved documents}\}|}$$

"fraction of retrieved documents that are relevant"

- Evaluate accuracy of top-K recommendations
- Penalizes recommending items user doesn't care about

Recall@K

$$\text{recall} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{relevant documents}\}|}$$

"fraction of relevant documents that were retrieved"

- Measures coverage of recommendations
- Avoids ranking on purely popularity

Hit Rate@K

$$\text{Hits@k} = \frac{|\{t \in \mathcal{K}_{test} \mid rank(t) \leq k\}|}{|\mathcal{K}_{test}|}$$

- Checks whether at least one relevant item is in top-K
- Binary check for model

- MSE/RMSE not included as they are metrics for rating prediction, not ranking
- AUC used for over/underfitting during training but not test set as it considers all pairwise score, not top-K ranking performance

4. Evaluation

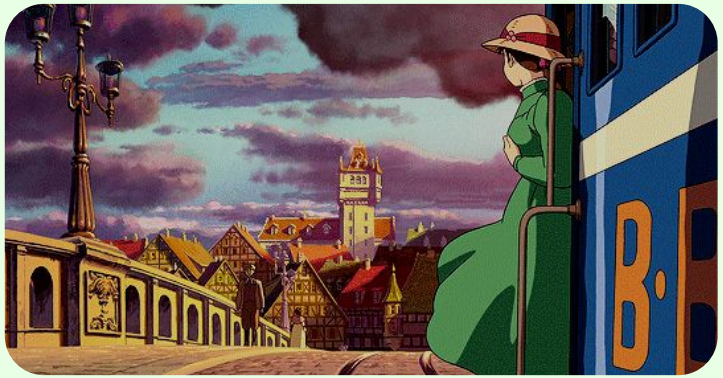
What are some baselines (trivial or otherwise) for your task? How do you demonstrate that your method is better than these methods?

We discussed two simple baseline models:

- **1. Popularity-Based & User/Item Average Model**
- **2. Item-Based Collaborative Filtering**

We discussed 3 evaluation metrics:

- **1. Precision@K**
- **2. Recall@K**
- **3. Hit Rate@K**



Expected Hybrid BPR Improvements

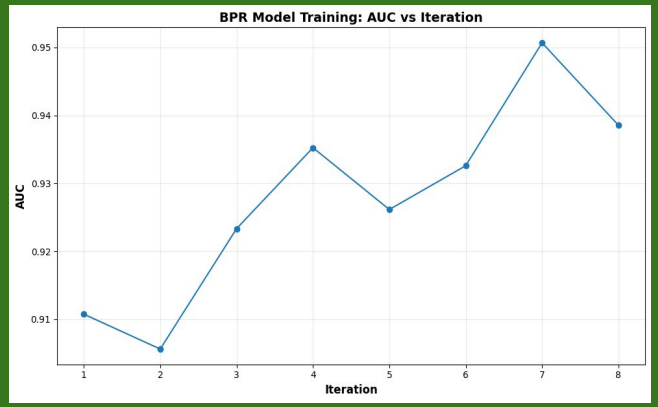
- Can capture hidden relationships with latent dimensions
- Higher recall for less popular anime
- Can personalize recommendations to users

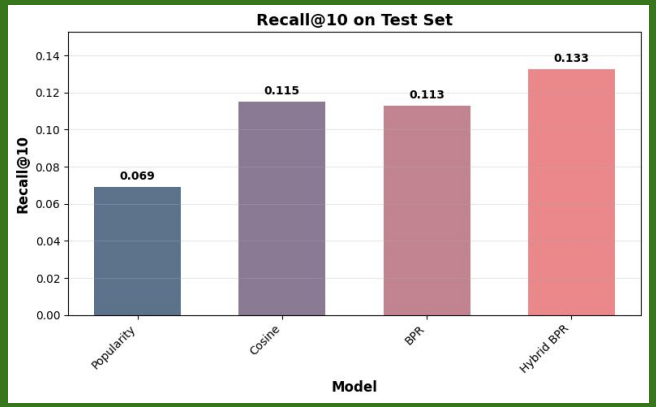
- Can capture hidden relationships with latent dimensions
- Better at cold start, can rank anime for users with sparse data

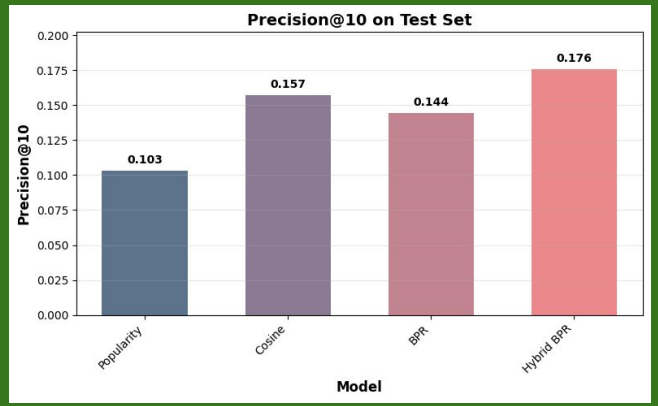
4. Evaluation

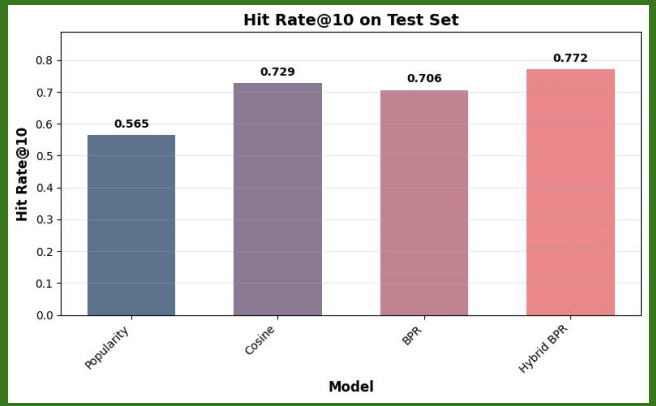
Walk through the implementation of your evaluation protocol, and support your evaluation with tables, plots, statistics, etc.









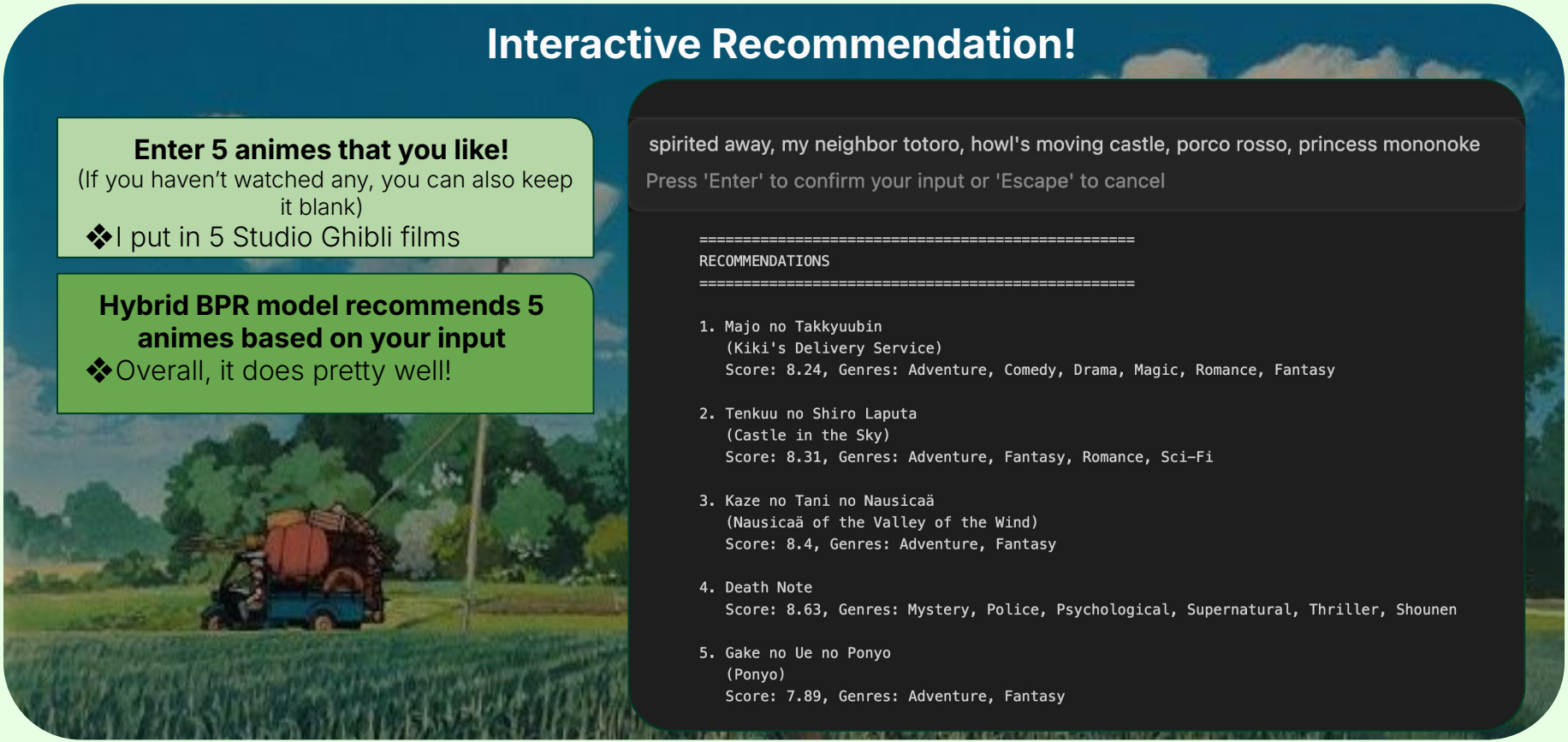


4. Evaluation

Interactive Recommendation!

Enter 5 animes that you like!
(If you haven't watched any, you can also keep it blank)
❖I put in 5 Studio Ghibli films

Hybrid BPR model recommends 5 animes based on your input
❖Overall, it does pretty well!



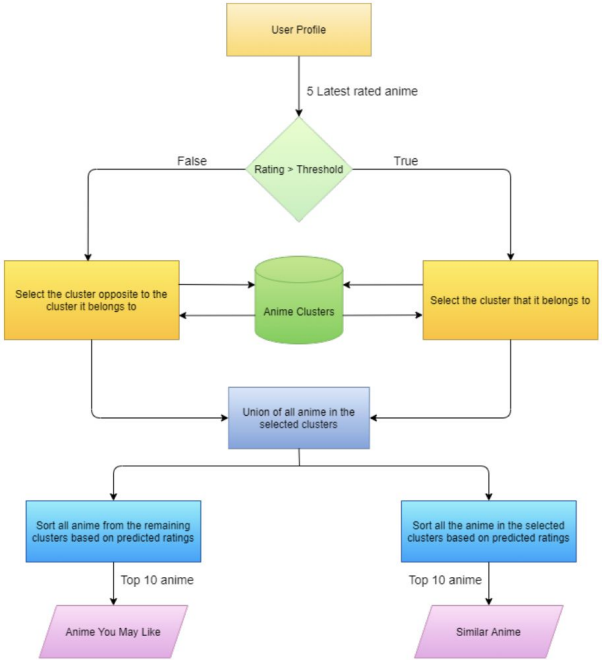
How has this dataset (or similar datasets) been used before?

How has prior work approached the same (or similar) tasks?

How do your results match or differ from what has been reported in related work?

5. Discussion of Related Work

Soni, B., Thakuria, D., Nath, N., Das, N., & Boro, B. (2021). *RikoNet: A novel anime recommendation engine*.



Purpose

- Predict ratings for anime that a user has not rated yet, and then use those predictions to find similar animes to ones the user likes/dislikes

Approach

- One autoencoder predict ratings for unrated items for a user
- Another autoencoder uses top users (by number of interactions) and creates embeddings for animes using metadata and ratings
- Group similar animes into clusters and recommend animes based on cluster similarity

Similarities with My Approach:

- Hybrid models that use collaborative filtering and item metadata
- Provide top-K recommendation based on user likes/dislikes

Differences with My Approach:

- Use autoencoder for collaborative filtering
- Work calculates ratings rather than ranking
- Clustering by item embeddings

Method	MSE (lower is better)	RMSE (lower is better)
Global Average	1.129	1.062
User Average	0.929	0.964
User KNN – Cosine	0.813	0.813
Item KNN - Cosine	0.854	0.862
User KNN – Pearson	0.814	0.902
Item KNN - Pearson	0.796	0.892
SVD	0.879	0.937
Biased MF	0.768	0.877
SVD++	0.756	0.869
DNNRec Kiran et al. (2020)	0.747	0.864
UBP Su et al. (2021)	0.724	0.851
PDMF-post 50D Yuan et al. (2021)	0.427	0.654
Proposed Model: RikoNet	**0.349**	**0.591**

Reference link: https://doi.org/10.48550/arXiv.2106.12970

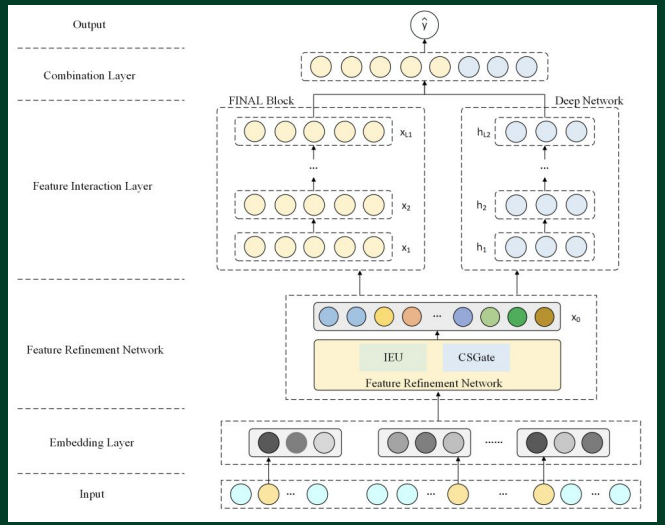
How has this dataset (or similar datasets) been used before?

How has prior work approached the same (or similar) tasks?

How do your results match or differ from what has been reported in related work?

5. Discussion of Related Work

Hou, W. et al. (2021). *Research on Anime Recommendation Algorithm Based on Parallel Feature Interaction*



- One network captures high-order interactions between users and anime
- Other adjusts weights of user features based on context

Purpose

- Personalized anime recommendation algorithm based on “parallel feature interaction”
- Model relationship between users and anime with item features

Approach

- Parallel feature interaction: several neural network combined to learn interactions between users and items
- Combine outputs from parallel networks to output a likelihood score of user liking the input animes

Similarities with My Approach:

- Both models use interaction and item metadata
- Final model is a combination of multiple sub-models

Differences with My Approach:

- Work doesn’t use interaction matrix to avoid computation complexity
- Takes into account temporal data

Model Class	Datasets / Model	Avazu AUC	Avazu Logloss	Frappe AUC	Frappe Logloss	MovieLens AUC	MovieLens Logloss	Anime(Ours) AUC	Anime(Ours) Logloss
First-Order	LR	0.7516	0.3735	0.9356	0.3076	0.9342	0.3201	0.8326	0.4729
Second-Order	FM	0.7613	0.3677	0.9671	0.2065	0.9434	0.2729	0.8396	0.4673
	AFM	0.7574	0.3705	0.9697	0.2264	0.9472	0.2653	0.8454	0.4606
	FwFM	0.7602	0.3688	0.9776	0.2030	0.9558	0.2426	0.8463	0.4586
	FmFM	0.7595	0.3689	0.9749	0.2004	0.9465	0.2714	0.8443	0.4590
High-Order	Wide&Deep	0.7649	0.3665	0.9841	0.1490	0.9688	0.2161	0.8495	0.4506
	DeepFM	0.7648	0.3667	0.9842	0.1482	0.9685	0.2130	0.8510	0.4492
	DCN	0.7652	0.3665	0.9839	0.1527	0.9687	0.2160	0.8512	0.4496
	xDeepFM	0.7647	0.3671	0.9845	0.1466	0.9697	0.2409	0.8514	0.4496
	AutoInt+	0.7645	0.3668	0.9848	0.1490	0.9692	0.2148	0.8485	0.4522
	AFN+	0.7640	0.3671	0.9826	0.2139	0.9642	0.3030	0.8496	0.4507
	EDCN	0.7652	0.3670	0.9850	0.1547	0.9671	0.2122	0.8472	0.4532
	MaskNet	0.7643	0.3674	0.9837	0.1696	0.9672	0.2364	0.8500	0.4520
	EulerNet	0.7645	0.3672	0.9848	0.1313	0.9703	0.1998	0.8520	0.4504
	Ours	0.7658	0.3662	0.9852	0.1434	0.9707	0.2116	0.8530	0.4478

Reference link: https://ieeexplore.ieee.org/document/10863921

6. Summary

Prediction Task

Top-K personalized anime recommendation:
Given a user's historical interactions, rank all unseen anime and return the top-K items the user is most likely to enjoy.

Model

Hybrid BPR Model:
Content-based and collaborative filtering approach that uses weighted combination of popularity/user interaction, cosine similarity, implicit BPR, and anime metadata to generate scores for each anime and user, which are then used for ranking.

Key Results

Best AUC (Validation Set)	**0.95**
Precision@K=10 (Test Set)	**0.176**
Recall@K=10 (Test Set)	**0.133**
Hit Rate@K=10 (Test Set)	**0.772**

Thank you!

